Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems

Improving Sim-to-Real transfer of Learning-based Policies for Quadcopter Racing through Domain Randomization

Semestral Project

Adam El Ghaib

Prague, January 2025

Study programme: Cybernetics and Robotics (Space Master)
Branch of study: "ask"

Supervisor: Ing. Robert Pěnička, Ph.D.

Acknowledgments

Firstly, I would like to express my gratitude to my supervisor.

Swap this for the Thesis Assignment, when you have it from the department.

Declaration

I declare that presented work was developed independently, and that I have listed all sources of information used within, in accordance with the Methodical instructions for observing ethical principles in preparation of university theses.

Date ....

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abstrakt

Výzkum na poli autonomních bezpilotních prostředků (UAV) se stal významným oborem mobilní robotiky.

Klíčová slova Bezpilotní Prostředky, Automatické Řízení

Abbreviations

UAV Unmanned Aerial Vehicle

RL Reinforcement Learning

DR Domain Randomization

POMDP Partially Observable Markov Decision Process

DoF Degrees of Freedom

PPO Proximal Policy Optimization

AC Actor Critic

MLP Multi-Layer Perceptron

CTBR collective thrust and body rates

Contents

1 Introduction

In the context of drone racing, where quadcopters are pushed to their physical and computational limits, there is a growing trend toward using neural networks for control. As demonstrated in [2], this approach has made it possible to outperform some of the world's most skilled human pilots.

These neural networks controllers are tipically optimized using RL algorithms. However, despite many of the advantadges that RL bring, it is still a hilghly sample inefficent algorithm that makes collecting data in the real world dangerous an expensive. Because of this, training is often performed in simulated environments that provide a safe and scalable alternative. Nonetheless, this solution comes with other problems since the resulting policies, learned in simulations, often perform worse than expected or fail to transfer to real-world scenarios altogether. This discrepancy in the policy's performance between simulation and reality is known as the Sim2Real gap, and it mainly arises from an undesired overfit of the optimization process to the simulated scenario.

1.1 Related works

Demonstrating that Domain Randomization (DR) can significantly increase the adaptability and robustness of autonomous systems, the work of Ferede et al. [1] serves as the direct inspiration for this project. Their research showed that a DR-trained neural network controller could successfully generalize across various quadcopter platforms.

1.2 Contributions

(Note: rather than contribute this work aims to reproduce the results from [1])

1.3 Mathematical notation

It is a good practice to define basic mathematical notation in the introduction. See Table 1.1 for an example.

$\mathbf{x}$, $\boldsymbol{\alpha}$	vector, pseudo-vector, or tuple
$\hat{\mathbf{x}}$, $\hat{\boldsymbol{\omega}}$	unit vector or unit pseudo-vector
$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3$	elements of the *standard basis*
$\mathbf{X}, \boldsymbol{\Omega}$	matrix
$\mathbf{I}$	identity matrix
$x = \mathbf{a}^\intercal \mathbf{b}$	inner product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \times \mathbf{b}$	cross product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \circ \mathbf{b}$	element-wise product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x}_{(n)} = \mathbf{x}^\intercal \hat{\mathbf{e}}_n$	n^{th} vector element (row), $\mathbf{x}, \mathbf{e} \in \mathbb{R}^3$
$\mathbf{X}_{(a,b)}$	matrix element, (row, column)
x_d	x_d is *desired*, a reference
$\dot{x}, \ddot{x}, \dddot{x}, \ddddot{x}$	1^{st}, 2^{nd}, 3^{rd}, and 4^{th} time derivative of x
$x_{[n]}$	x at the sample n
$\mathbf{A}, \mathbf{B}, \mathbf{x}$	LTI system matrix, input matrix and input vector
SO(3)	3D special orthogonal group of rotations
SE(3)	*SO(3)* $\times \mathbb{R}^3$, special Euclidean group

Table 1.1: Mathematical notation, nomenclature and notable symbols.

2 Methodology

2.1 Problem definition

We define a domain as a specific subset of possible Partially Observable Markov Decision Process (POMDP) environments. While a complete POMDP is formally characterized by an 8-tuple **<empty citation>** we focus on a reduced 6-tuple representation that captures the essential elements for our domain definition. A standard domain is characterized by $\mathcal{D} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R)$ where:

- $\mathcal{S}$: denotes a set of states called the state space.
- $\mathcal{A}$: is the set of actions called the action space.
- $\mathcal{O}$: is the set of observations called the observation space.
- $\mathbf{P}(\mathbf{s}'|\mathbf{s},\mathbf{a})$: is the probability that a given action $a \in A$, at state $s \in S$ will lead to a new state $s' \subset S$.
- $\mathbf{O}(\mathbf{o}|\mathbf{s}',\mathbf{a})$: is the observation probability, representing the probability of observing $o \in \mathcal{O}$ after taking action a and transitioning to state s'.
- $\mathbf{R}(\mathbf{s}',\mathbf{s},\mathbf{a})$: is the immediate reward, or expected inmediate reward, received for transitioning from s to s' after performing action a.

From the source domain, $\mathcal{D}_s$, (i.e. simulation), we can control a set of N parameters of the variable aspects of each environment that define a configuration vector, $\boldsymbol{\xi} = [\xi_i, \xi_{i+1}, ..., \xi_N]$. Examples of ξ_i can include:

Physical properties	ξ_{mass}, ξ_{inertia}, ξ_{drag}, ξ_{gravity}
Perception properties	$\xi_{\text{sensor\_noise}}$, $\xi_{\text{sensor\_latency}}$
Viual properties	ξ_{lighting}, ξ_{texture}, ξ_{fov}
Scene properties	$\xi_{\text{gate\_pos}}$, $\xi_{\text{gate\_size}}$, , $\xi_{\text{gate\_num}}$

Each configuration is sampled from a randomization space, $\boldsymbol{\xi} \in \Xi \subset \mathbb{R}^N$, with a probability distribution $\boldsymbol{\xi} \sim p(\boldsymbol{\xi})$. Where the parameters are bounded to be within an interval $\xi_i \in [\xi_i^{\text{low}}, \xi_i^{\text{high}}]$.

At the start of every training episode, a new environment insance is created by sampling a new configuration vector $\boldsymbol{\xi}$ from the distribution $p(\boldsymbol{\xi})$. Therefore, the agent will interact with a specific POMDP environment, that is to say, a specific domain $\mathcal{D}_{\boldsymbol{\xi}}$. This way, the agent can collect data from a vareity of environments that otherwise would reamain unexplored.

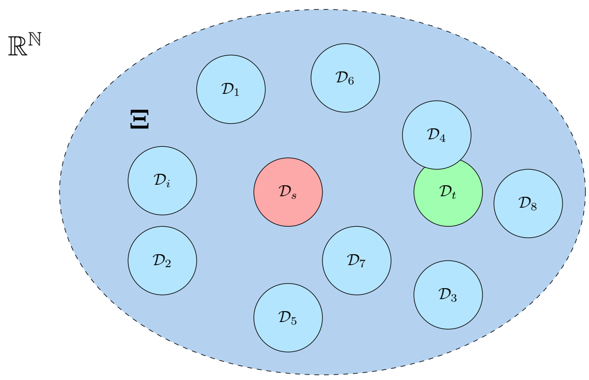

Figure 2.1: Red and green the source and target domains respectively. In blue, differents instances of the randomized source space (all represented domains with the same size, N).

Hereby, during training, the neural network weights and biases θ are optimized to maximize the expected reward R **across a distribution of randomized domains** [0]. With the aim of finding the optimal, θ^* that will lead to the optimal parametrized policy π^*_θ.

$$\theta^* = \arg\max_{\theta} \mathbb{E}_{\boldsymbol{\xi}\sim\Xi}\left[\mathbb{E}_{\pi_\theta,\tau\sim D_\xi}[R(\tau)]\right] \tag{2.1}$$

Where $R(\tau) = \sum_{t=0}^{n} \gamma^t r_t$ is the cummulative discounted reward over a trajectory (or sequence) of states and actions $\tau = (s_0, a_0, s_1, a_1, ...)$, for the episode length n.

As illustrated in Figure 2.1 and described in Equation 2.1, the policy is exposed to a variety of domains to promote generalization and improve robustness, thereby increasing the likelihood of successful transfer to the real environment. However, this strategy further reduces the sample efficiency of RL and increases training time. Therefore, it is essential to properly bound the randomization space Ξ so that it encompasses only the minimum necessary range required to include the real domain.

2.2 Quadcopter model

In the simulated environments, the quadcopter is modeeld as a 6 Degrees of Freedom (DoF), rigid body. The state of the quadcopter is defined by its position $\mathbf{p}$, velocity $\mathbf{v}$, orientation $\mathbf{q}$, body rates $\boldsymbol{\omega}$ and rotor speeds $\boldsymbol{\Omega}$. Namely, $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}]^T$. The position, velocity and acceleration are respresented in the world frame $\mathcal{W} \in \mathbb{R}^3$, while the body rates in the body frame $\mathcal{B} \in \mathbb{R}^3$. The orientation is described by an unit quaternion rotation $\mathbf{q} \in \mathbb{SO}(3)$. Hence, the kinematics and dynamics of the drone are:

$$\dot{\mathbf{p}}^{\mathcal{W}} = \mathbf{v}^{\mathcal{W}} \tag{2.2}$$

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\mathbf{q} \odot \begin{bmatrix}0 & \boldsymbol{\omega}^{\mathcal{B}}\end{bmatrix}^T \tag{2.3}$$

$$\dot{\mathbf{v}}^{\mathcal{W}} = \frac{R(\mathbf{q})}{m}\left(\mathbf{f}_T^{\mathcal{B}} + \mathbf{f}_D^{\mathcal{B}}\right) + \mathbf{g}^{\mathcal{W}} \tag{2.4}$$

$$\dot{\boldsymbol{\omega}}^{\mathcal{B}} = J^{-1}\left(\boldsymbol{\tau}^{\mathcal{B}} - \boldsymbol{\omega}^{\mathcal{B}} \times J\boldsymbol{\omega}^{\mathcal{B}}\right) \tag{2.5}$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_{\mathrm{m}}}\left(\boldsymbol{\Omega}_c - \boldsymbol{\Omega}\right) \tag{2.6}$$

Where, $\odot$ represents quaternion multiplication, $R(\mathbf{q})$ is the rotational matrix from body frame $\mathcal{B}$ to world frame $\mathcal{W}$ at orientation $\mathbf{q}$, J is the diagonal inertia matrix, $\mathbf{f}_T^{\mathcal{B}}$ and $\mathbf{f}_D^{\mathcal{B}}$ are the collective thrust and drag respectively, k_m the motor time constant, m the qudcopter's mass and $\mathbf{g}$ is gravity.

In the body frame $\mathcal{B}$, the individual motor thrusts $T_i,\ i = 1,..,4$, are assumed to be perpendicular to the $\mathbf{xy}$ plane. The collective thrust $\mathbf{f}_T^{\mathcal{B}}$, and torques $\boldsymbol{\tau}^{\mathcal{B}}$, can be maped from the individual motor thrusts with the allocation matrix Γ.

$$\begin{bmatrix}\mathbf{f}_T^{\mathcal{B}} \\ \tau_x \\ \tau_y \\ \tau_z\end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -d/\sqrt{2} & d/\sqrt{2} & d/\sqrt{2} & -d/\sqrt{2} \\ -d/\sqrt{2} & d/\sqrt{2} & -d/\sqrt{2} & d/\sqrt{2} \\ c_{tf} & c_{tf} & c_{tf} & c_{tf} \end{bmatrix} \begin{bmatrix}T_1 \\ T_2 \\ T_3 \\ T_4\end{bmatrix} \tag{2.7}$$

Being d the arm length and c_{tf} the linear torque constant. Likewise, the indidual motror thrusts T_i, can be mapped from the rotor speeds Ω_i with the thrust map coefficents, $\mathbf{k_T} = [a_0,\ a_1,\ a_2]$ that are determined experimentally fitting a quadratic curve.

$$T_i = \mathbf{k_T}\begin{bmatrix}\Omega_i^2 \\ \Omega_i \\ 1\end{bmatrix} = \begin{bmatrix}a_2 & a_1 & a_0\end{bmatrix}\begin{bmatrix}\Omega_i^2 \\ \Omega_i \\ 1\end{bmatrix} \tag{2.8}$$

The total drag force, $\mathbf{f}_D^{\mathcal{B}}$ is calculated using a simplified aerodynamic model function of the dron's linear velocity and proportional to the drag coefficents.

$$\mathbf{f}_D^{\mathcal{B}} = \begin{bmatrix} C_{D_x} & 0 & 0 \\ 0 & C_{D_y} & 0 \\ 0 & 0 & C_{D_z} \end{bmatrix} \begin{bmatrix} v_x^{\mathcal{B}} \\ v_x^{\mathcal{B}} \\ v_x^{\mathcal{B}} \end{bmatrix} \tag{2.9}$$

2.3 RL Set-Up

Although the simplified quadcopter model defined in 2.2 is used to represent the drone dynamics in simulation, the RL setup employed in this project is **model-free**. Specifically, it consists of an Actor Critic (AC) structure optimized using Proximal Policy Optimization (PPO) **<empty citation>** The implementation relies on Stable-Baselines3 **<empty citation>** to interface with a custom C++ Gymnasium environment (based on flightmare **<empty citation>**) and to train the AC networks using PPO.

The actor policy netwrok π_θ, is defined by a Multi-Layer Perceptron (MLP) that takes an observation vector at time step t, Equation 2.10 of 31 inputs. The observation vector is sampled from a bounded observation space $o_t \in \mathcal{O} \subset \mathbb{R}^{31}$.

$$o_t = \begin{bmatrix} \mathbf{p}_t & \mathbf{v}_t & \mathrm{vec}R(\mathbf{q})_t & u_{t-1} & \mathbf{p_g}_t^{TL} & \mathbf{p_g}_t^{TR} & \mathbf{p_g}_t^{BL} & \mathbf{p_g}_t^{BL} & \mathbf{p}_t^{\mathrm{look}} \end{bmatrix}^T \tag{2.10}$$

Where u_{t-1} is the throttle motor command at the prevous time step. $\mathbf{p_g}_t^X$ are relative positions of the corners of the target gate at time t with respect the body frame $\mathcal{B}$. And, $\mathbf{p}_t^{\mathrm{look}}$ is the look at position of the target gate at time t, also in the body frame. The input layer is followed by three hidden layers of 128 neurons each, and it outputs 4 neurons. The outputs are the collective thrust and body rates (CTBR), sampled from a bounded action space $a_t \in \mathcal{A} \subset \mathbb{R}^4$.

$$a_t = \begin{bmatrix} \mathbf{f}_{Tt} & \boldsymbol{\omega}_t \end{bmatrix}^T \tag{2.11}$$

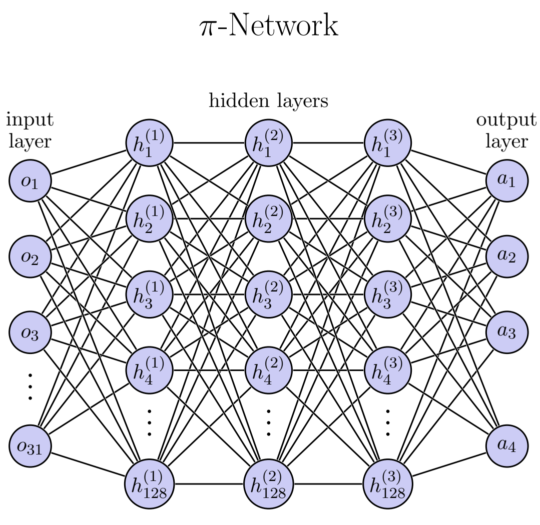

Figure 2.2: Actor network architecture

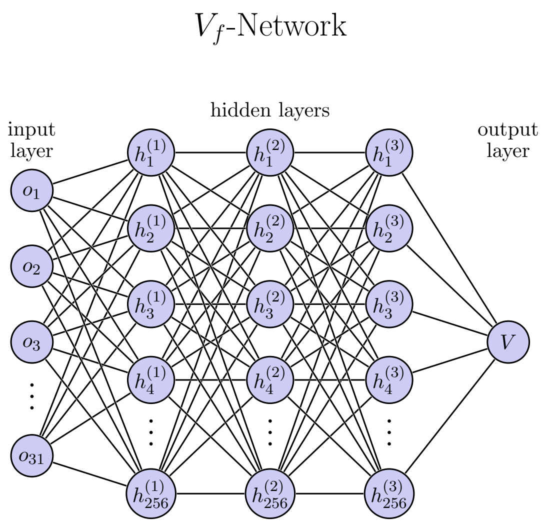

Figure 2.3: Critic network architecture

On the other hand, the critic network aritecthure V_f, is formed also by 3 hidden layers but of 256 neurons each, it has the same input space and it oputs a single value based on the observation o_t. The hyperparameters used for tranning can be found in Appendix **??**

2.4 DR configurations

The first randomiztion configuration condsidered has been to randomize the following dynamical parameters:

$$\boldsymbol{\xi}_d = [m, J, C_D, k_m, \mathbf{k_T}, \Gamma]$$

Parameter	Nominal Value	Randomization Distribution
m [kg]	0.916	$\pm\mathcal{U}(0, 0.1)$
$J_{xx},\ J_{yy}, J_{zz}$ [kg]	0.004, 0.008, 0.011	$\cdot\mathcal{U}(0.8, 1.2)$
$C_{D_x},\ C_{C_y}, C_{D_z}$	0.38, 0.47, 1.51	$\pm\mathcal{U}(0, 0.14), \pm\mathcal{U}(0, 0.2), \pm\mathcal{U}(0, 0.5)$
k_m [kg]	0.916	$\pm\mathcal{U}(0, 0.1)$
a_2, a_1, a_0 [kg]	0.916	$\pm\mathcal{U}(0, 0.1)$
m [kg]	0.916	$\pm\mathcal{U}(0, 0.1)$

Table 2.1: Desc

To Do's

- Explain that ideally Ξ should be similar to Θ. DR worsens sample efficency but improves robustness and adaptation
- Explain that MDP is acually POMDP and that the randomization configurations only include the transition probability P, and maybe observation? Well acually it will be observation memory right?
- Explain what trajectory is, reward is,

3 Results

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

4 Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

5 References

[1] R. Ferede, T. Blaha, E. Lucassen, C. De Wagter, and G. C. H. E. de Croon, "One net to rule them all: Domain randomization in quadcopter racing across different platforms," 2025. DOI: 10.48550/ARXIV.2504.21586. [Online]. Available: https://arxiv.org/abs/2504.21586.

[2] T. Baca, M. Petrlik, M. Vrba, *et al.*, "The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 26, pp. 1–28, 1 May 2021.

[0] L. Weng, "Domain randomization for sim2real transfer," *lilianweng/github/io*, 2019. [Online]. Available: https://lilianweng.github.io/posts/2019-05-05-domain-randomization/.

A Appendix A